

초급반 6주차

이O재 @ AIKor, CYDF



목차

- 복습
- 포인터
- 동적할당
- 파일입출력
- 연습문제

복습 - 1912

```
#include <bits/stdc++.h>

int fac(int n) {
    if (n == 1) return 1;
    return n * fac(n - 1);
}

int main() {
    int n;
    scanf("%d", &n);
    printf("%d\n", fac(n));
}
```

복습 - 1920

```
#include <bits/stdc++.h>

void bin(int n) {
    if (n == 0) return;
    bin(n / 2);
    printf("%d", n % 2);
}

int main() {
    int n;
    scanf("%d", &n);
    bin(n);
}
```

복습 - 3420

```
#include <bits/stdc++.h>

void hanoi(int disk, int from, int to, int tmp) {
    if (disk == 1) {
        printf("Disk %d: %c to %c\n", disk, from, to);
        return;
    }

    hanoi(disk - 1, from, tmp, to);
    printf("Disk %d: %c to %c\n", disk, from, to);
    hanoi(disk - 1, tmp, to, from);
}

int main() {
    int n;
    scanf("%d", &n);
    hanoi(n, 'A', 'C', 'B');
}
```

포인터

- 포인터 연산자
 - &: 주소
 - *: 주소가 가리키는 값

```
&a      // a의 주소
*(&a)    // a의 주소가 가리키는 값 = a의 값

p = &a  // a의 주소를 p에 저장한다
*p      // p가 가리키는 값 = a의 값
```

포인터

- 변수에 값을 저장할 때 컴퓨터의 메모리에 저장됨

```
#include <bits/stdc++.h>
```

```
int main() {  
    int a = 10;  
    printf("%p\n", &a);  
}
```

```
0x16f7a70cc
```

포인터

- 포인터 변수를 이용해서 실제 저장되어 있는 값을 확인할 수 있음

```
#include <bits/stdc++.h>

int main() {
    int a = 10;
    printf("%p\n", &a);

    int* p;
    scanf("%x", &p);
    printf("%d", *p);
}
```

```
0x16f8cb0cc
0x16f8cb0cc
10
```

포인터

- 변수와 포인터 변수

```
#include <bits/stdc++.h>

int main() {
    int a = 10;
    printf("%p\n", &a);

    int* p = &a;
    printf("%d", *p);
}
```

0x1 6f8c b0a0	?	?	?	?	?	?	?	
0x1 6f8c b0a8	?	?	?	?	?	?	?	
0x1 6f8c b0b0	?	?	?	?	?	?	?	
0x1 6f8c b0b8	?	?	?	?	?	?	?	
0x1 6f8c b0c0	cc	b0	8c	6f	01	00	00	p
0x1 6f8c b0c8	?	?	?	?	0a	00	00	a
0x1 6f8c b0d0	?	?	?	?	?	?	?	
0x1 6f8c b0d8	?	?	?	?	?	?	?	
0x1 6f8c b0e0	?	?	?	?	?	?	?	

- stack 형태로 저장되기 때문에, 나중에 선언된 변수는 낮은 주소에 위치
- 8byte (64 bit) aligned

포인터

- 변수와 포인터 변수

```
p = 0x0000 0001 6f8c b0cc  
&p = 0x0000 0001 6f8c b0c0  
*p = 10  
  
a = 10  
&a = 0x0000 0001 6f8c b0cc
```

```
0x1 6f8c b0a0  
0x1 6f8c b0a8  
0x1 6f8c b0b0  
0x1 6f8c b0b8  
0x1 6f8c b0c0  
0x1 6f8c b0c8  
0x1 6f8c b0d0  
0x1 6f8c b0d8  
0x1 6f8c b0e0
```

?	?	?	?	?	?	?	?
?	?	?	?	?	?	?	?
?	?	?	?	?	?	?	?
?	?	?	?	?	?	?	?
cc	b0	8c	6f	01	00	00	00
?	?	?	?	0a	00	00	00
?	?	?	?	?	?	?	?
?	?	?	?	?	?	?	?

p

a

포인터

- 포인터 변수의 할당

```
#include <bits/stdc++.h>

int main() {
    // int* numPtr = 0x100; --> error
    int* numPtr = (int *) 0x100;
    printf("%d\n", *numPtr);
}
```

Process finished with exit code 139 (interrupted by signal 11:SIGSEGV)

포인터

- 사용할 주소는 할당되어 있음 → 0x100은 범위 밖이므로 에러

```
pwndbg> vmmap
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA

    Start                End Perm    Size  Offset File (set vmmap-prefer-relpaths on)
0x555555554000          0x555555555000 r--p    1000      0 main
0x555555555000          0x555555556000 r-xp    1000    1000 main
0x555555556000          0x555555557000 r--p    1000    2000 main
0x555555557000          0x555555558000 r--p    1000    2000 main
0x555555558000          0x555555559000 rw-p    1000    3000 main
0x7ffff7c00000          0x7ffff7c28000 r--p   28000      0 /usr/lib/x86_64-linux-gnu/libc.so.6
0x7ffff7c28000          0x7ffff7dc1000 r-xp  199000   28000 /usr/lib/x86_64-linux-gnu/libc.so.6
0x7ffff7dc1000          0x7ffff7e30000 r--p    6f000  1c1000 /usr/lib/x86_64-linux-gnu/libc.so.6
0x7ffff7e30000          0x7ffff7e34000 r--p    4000  22f000 /usr/lib/x86_64-linux-gnu/libc.so.6
0x7ffff7e34000          0x7ffff7e36000 rw-p    2000  233000 /usr/lib/x86_64-linux-gnu/libc.so.6
0x7ffff7e36000          0x7ffff7e43000 rw-p    d000      0 [anon_7ffff7e36]
0x7ffff7fa1000          0x7ffff7fa4000 rw-p    3000      0 [anon_7ffff7fa1]
0x7ffff7fb4000          0x7ffff7fb6000 rw-p    2000      0 [anon_7ffff7fb4]
0x7ffff7fb6000          0x7ffff7fba000 r--p    4000      0 [vvar]
0x7ffff7fba000          0x7ffff7fbc000 r--p    2000      0 [vvar_vclock]
0x7ffff7fbc000          0x7ffff7fbe000 r-xp    2000      0 [vdso]
0x7ffff7fbe000          0x7ffff7fbf000 r--p    1000      0 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
0x7ffff7fbf000          0x7ffff7fed000 r-xp   2e000   1000 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
0x7ffff7fed000          0x7ffff7ffb000 r--p    e000  2f000 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
0x7ffff7ffb000          0x7ffff7ffd000 r--p    2000   3d000 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
0x7ffff7ffd000          0x7ffff7ffe000 rw-p    1000   3f000 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
0x7ffff7ffe000          0x7ffff7fff000 rw-p    1000      0 [anon_7ffff7ffe]
0x7ffff7fff000          0x7ffff7fff000 rw-p    1000      0 [stack]
0xfffffffff60000      0xfffffffff601000 --xp    1000      0 [vsyscall]
```

포인터

- 포인터 변수의 연산

```
#include <bits/stdc++.h>

int main() {
    int a = 10, b = 20, c = 30;
    int *p = &a;

    printf("%p %p %p\n", &a, &b, &c);          // 0x16f3430c4 0x16f3430c0 0x16f3430bc
    printf("%p %p %p\n", p, p - 1, p - 2);    // 0x16f3430c4 0x16f3430c0 0x16f3430bc

    printf("%d\n", *p);                        // 10
    printf("%d\n", *(p - 1));                  // 20
    printf("%d\n", *(p - 2));                  // 30
}
```

포인터

- 포인터 변수의 연산

```
#include <bits/stdc++.h>

int main() {
    char a = 'A', b = 'B', c = 'C';
    char *p = &a;

    printf("%p %p %p\n", &a, &b, &c);          // 0x16fb630c7 0x16fb630c6 0x16fb630c5
    printf("%p %p %p\n", p, p - 1, p - 2);      // 0x16fb630c7 0x16fb630c6 0x16fb630c5

    printf("%c\n", *p);                         // A
    printf("%c\n", *(p - 1));                   // B
    printf("%c\n", *(p - 2));                   // C
}
```

포인터

- 배열에서의 포인터

```
#include <bits/stdc++.h>

int main() {
    int a[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    int* p = a;

    for (int i = 0; i < 10; i++) {
        printf("%d ", *(p + i));
    }
}
```

1 2 3 4 5 6 7 8 9 10

포인터

- 배열에서의 포인터

```
#include <bits/stdc++.h>

int main() {
    int a[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    // char *p = a; --> char*에 int*를 대입하는 것으로 에러
    char* p = (char *)a;

    for (int i = 0; i < 10; i++) {
        printf("%d ", *(p + 4 * i));
    }
}
```

1 2 3 4 5 6 7 8 9 10

포인터

- 포인터를 이용한 값 바꾸기

```
#include <bits/stdc++.h>

int main() {
    int a = 10;
    int* p = &a;
    *p = 20;
    printf("%d\n", a); // 20
}
```

포인터

- 함수에서의 포인터
- Call by Value: 함수가 인자로 전달받은 값을 복사하여 사용
- Call by Reference: 함수가 인자로 전달받은 값을 참조하여 사용

포인터

- Call by value

```
#include <bits/stdc++.h>

void swap(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
}

int main() {
    int a = 10, b = 20;
    swap(a, b);
    printf("%d %d\n", a, b);
}
```

포인터

- Call by Reference

```
#include <bits/stdc++.h>

void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int a = 10, b = 20;
    swap(&a, &b);
    printf("%d %d\n", a, b);
}
```

포인터

- Call by Value
 - 장점: 복사하여 처리하기 때문에 원래 값이 보존됨
 - 단점: 복사를 하면 할 수록 메모리가 계속 사용됨
- Call by Reference
 - 장점: 복사하지 않고 직접 참조를 하기 때문에 빠름
 - 단점: 직접 참조를 하기 때문에 원본 값이 바뀔 수 있음

포인터

- Call by Reference 예시

```
#include <bits/stdc++.h>

void findMinMax(int a[], int len, int* minval, int* maxval) {
    *minval = *maxval = a[0];
    for (int i = 1; i < len; i++) {           // sizeof(a) / sizeof(int)는 안됨. 왜?
        if (a[i] < *minval) *minval = a[i];
        if (a[i] > *maxval) *maxval = a[i];
    }
}

int main() {
    int a[10] = {5, 2, 9, 1, 5, 6, 7, 3, 4, 8};
    int minval, maxval;
    findMinMax(a, 10, &minval, &maxval);

    printf("%d %d\n", minval, maxval);
}
```

동적 할당

- 정적 할당: 기존에 사용하던 배열. 컴파일 단계에서 크기가 지정됨
- 동적 할당: 런타임 단계에서 크기를 자유롭게 늘리거나 줄임

```
#include <bits/stdc++.h>

int main() {
    int* a = (int *)malloc(sizeof(int) * 10);
    for (int i = 0; i < 10; i++) {           // sizeof(a) 마찬가지로 안됨
        a[i] = i;
        printf("%d ", a[i]);
    }
    free(a);
}
```

0 1 2 3 4 5 6 7 8 9

- free를 하지 않으면 memory leak 발생

동적 할당

- 다양한 동적 할당 함수

```
#include <bits/stdc++.h>

int main() {
    int* a = (int *)malloc(sizeof(int) * 5);
    // a = {?, ?, ?, ?, ?}

    memset(a, -1, sizeof(int) * 5);
    // a = {-1, -1, -1, -1, -1}

    a = (int *)realloc(a, sizeof(int) * 10);
    // a = {-1, -1, -1, -1, -1, ?, ?, ?, ?, ?}

    int* b = (int *)calloc(5, sizeof(int));
    // b = {0, 0, 0, 0, 0}
}
```

동적 할당

- realloc 특징

- 메모리를 재할당할 때, 재할당하는 크기가 남은 여유 공간을 초과할 경우 기존의 메모리를 복사한 후 새로 할당함

```
#include <bits/stdc++.h>

int main() {
    int* a = (int *)malloc(sizeof(int) * 5);           // 0x101d55a50
    printf("%p\n", a);

    a = (int *)realloc(a, sizeof(int) * 6);           // 0x101d55a50
    printf("%p\n", a);

    a = (int *)realloc(a, sizeof(int) * 10);          // 0x101d55a70
    printf("%p\n", a);
}
```

동적 할당

- 2차원 배열의 동적 할당 방법

```
#include <bits/stdc++.h>

int main() {
    int* arr = (int*)malloc(sizeof(int) * 3 * 4);
    for (int i = 0; i < 3; ++i) {
        for (int j = 0; j < 4; ++j) {
            arr[i * 4 + j] = i * 4 + j; // arr[i][j] = i * 4 + j
        }
    }
}
```

동적 할당

- 2차원 배열의 동적 할당 방법

```
#include <bits/stdc++.h>

int main() {
    int** arr = (int**)malloc(sizeof(int*) * 3);
    for (int i = 0; i < 3; i++) {
        arr[i] = (int*)malloc(sizeof(int) * 4);
    }

    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 4; j++) {
            arr[i][j] = i * 4 + j;
        }
    }
}
```

파일 입출력

- file pointer를 이용해서 파일에 접근, 읽기 및 쓰기가 가능함
- input.txt 파일에 “Hello, World!” 입력하는 코드

```
#include <bits/stdc++.h>

int main() {
    FILE* fp = fopen("input.txt", "w");

    fprintf(fp, "Hello, World!\n");
    // fputs("Hello, World!\n", fp);

    fclose(fp);
}
```

파일 입출력

- input.txt 파일에 적힌 값을 읽어오는 코드

```
#include <bits/stdc++.h>

int main() {
    FILE* fp = fopen("input.txt", "r");
    char line[1024];

    fscanf(fp, "%[^\n]s", line);
    // fgets(line, sizeof(line), fp);

    printf("%s", line);
    fclose(fp);
}
```

파일 입출력

- fgets와 fputs를 이용해서 터미널 창에서 입출력도 가능함

```
#include <bits/stdc++.h>

int main() {
    char line[1024];
    fgets(line, 1024, stdin);
    fputs(line, stdout);
}
```

파일 입출력

- file descriptor를 이용해서 파일을 열 수도 있음
 - file descriptor: 프로세스에서 열린 파일 목록을 관리하기 위한 테이블 인덱스
- 일반 정규 파일(.txt 등), 소켓 등은 전부 file descriptor로 관리됨

```
#include <bits/stdc++.h>
#include <fcntl.h> // For open()
#include <unistd.h> // For close()

int main() {
    int fd = open("input.txt", O_RDONLY);
    printf("input.txt fd: %d\n", fd);
    if (fd == -1) { // 파일이 존재하지 않거나 권한 부족이면 -1
        perror("Failed to open input.txt");
        return 1;
    }
    close(fd);
}
```

파일 입출력

- file descriptor가 3부터 시작하는 이유
 - 0: stdin (표준 입력)
 - 1: stdout (표준 출력)
 - 2: stderr (표준 에러)

파일 입출력

- file descriptor의 관리

```
#include <bits/stdc++.h>
#include <fcntl.h> // For open()
#include <unistd.h> // For close()

int main() {
    int fd = open("input.txt", O_RDONLY);
    printf("input.txt fd: %d\n", fd);        // input.txt fd: 3
    close(fd);

    int fd2 = open("output.txt", O_WRONLY);
    printf("output.txt fd: %d\n", fd2);     // output.txt fd: 3
    close(fd2);
}
```

연습 문제

- 1405, 1468, 1476, 1484 동적 할당으로 해결하기